

LAB # 13

Exercise:

1. Use <https://reqres.in/api> and create Testcase to execute Get, Put, Patch, Post and delete requests also use variable to define url. /a

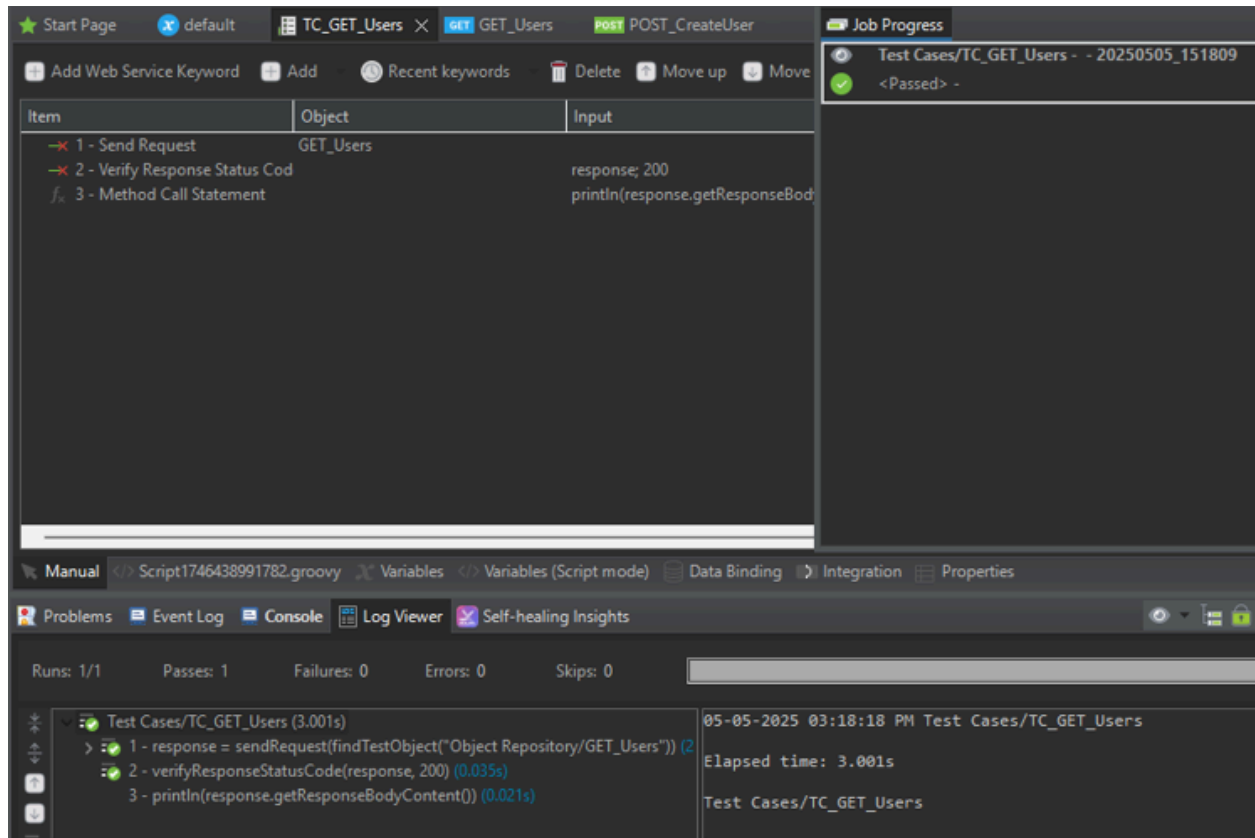
GET Request

The screenshot shows the Postman application interface. The top bar displays several tabs: GET_Users, DELETE_User, TC-DELETE_Users, PUT_UpdateUser, TC_PUT_Upda..., and RullAllTestC... The main workspace is divided into two panels. The left panel shows a GET request to the URL `${baseUrl}/users?page=2`. Below the URL bar, there are checkboxes for 'Follow redirects', 'Customize API methods', and 'Set timeout and response size limit'. A 'Query Parameters' section is expanded, showing a table with one parameter: 'page' with a value of '2'. At the bottom of the left panel, there is a 'Variables' tab with a 'Variables Editor' showing a table with one variable: 'baseUrl' of type 'String' with a default value of 'https://reqres.in/api'. The right panel shows the 'Response' section with a status of '200 OK', elapsed time of '193 ms', and size of '2 KB'. The response body is displayed in a pretty-printed JSON format:

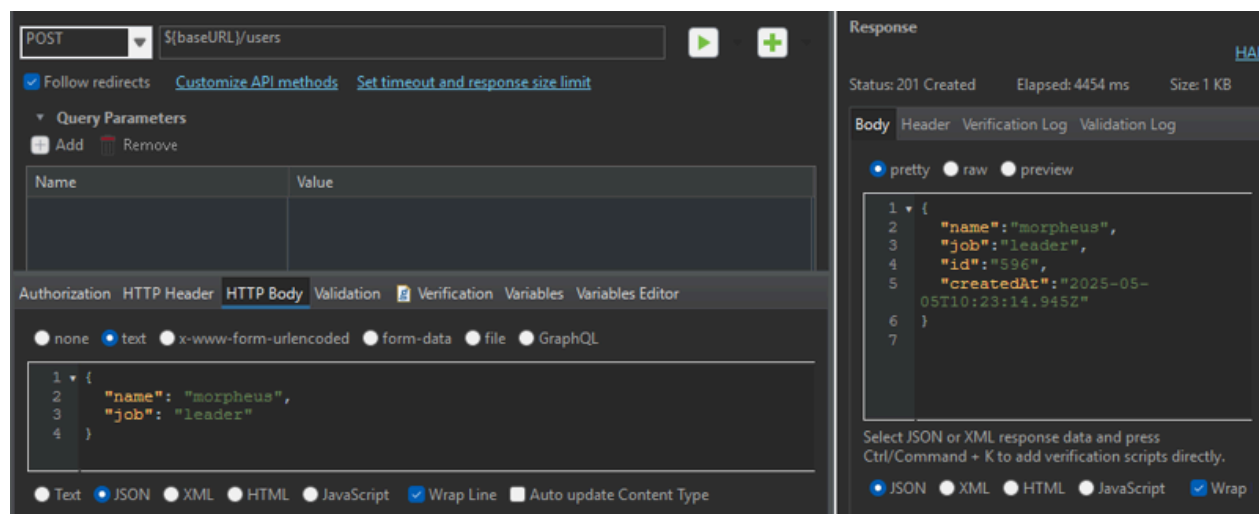
```
1 {
2   "page": 2,
3   "per_page": 6,
4   "total": 12,
5   "total_pages": 2,
6   "data": [
7     {
8       "id": 7,
9       "email": "michael.lawson@reqres.in",
10      "first_name": "Michael"
11    }
12  ]
13 }
```

Below the JSON response, there is a prompt: 'Select JSON or XML response data and press Ctrl/Command + K to add verification scripts directly.' At the bottom right, there are radio buttons for 'JSON', 'XML', 'HTML', and 'JavaScript', and a checked checkbox for 'Wrap'.

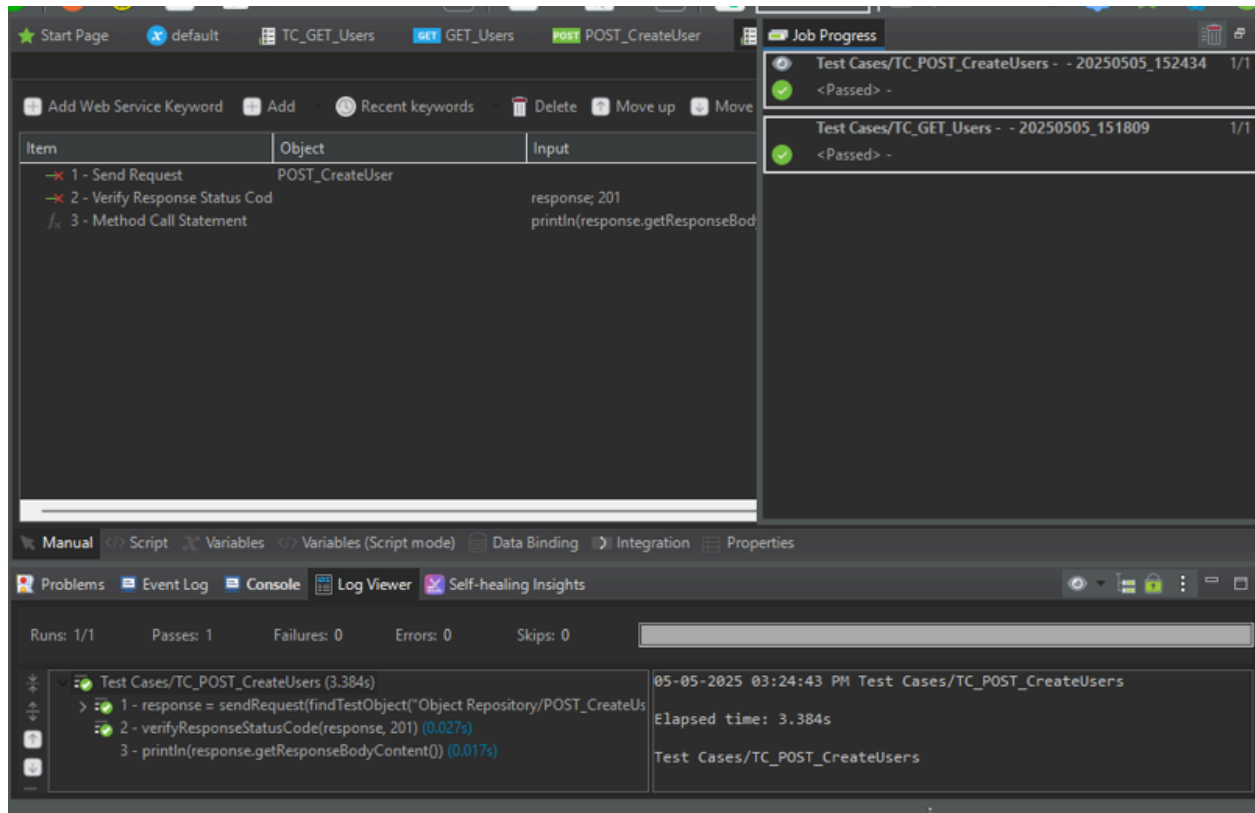
GET_TestCase



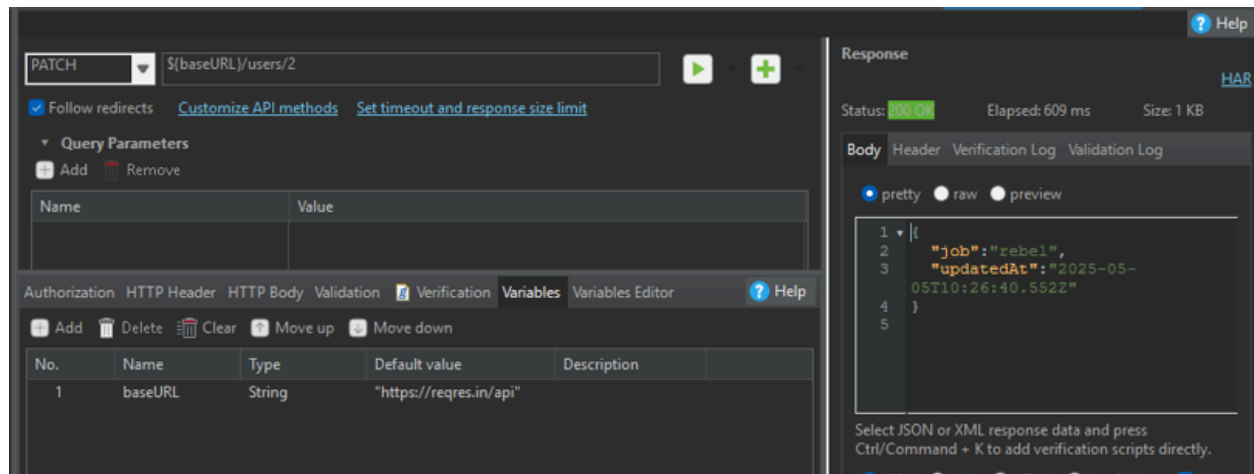
Post Request



POST_TestCase



Patch Request



default TC_GET_Users GET_Users POST POST_CreateUser TC_POST_Cre... Job Progress

Add Web Service Keyword Add Recent keywords Delete Move up Move

Item	Object	Input
1 - Send Request	PATCH_UpdateUser	
2 - Verify Response Status Cod		response; 200
3 - Method Call Statement		println(response.getResponseBod

Test Cases/TC_PATCH_UpdateUsers - - 20250505_152747 1/1
<Passed> -

Test Cases/TC_POST_CreateUsers - - 20250505_152434 1/1
<Passed> -

Test Cases/TC_GET_Users - - 20250505_151809 1/1
<Passed> -

Manual Script Variables Variables (Script mode) Data Binding Integration Properties

Problems Event Log Console Log Viewer Self-healing Insights

Runs: 1/1 Passes: 1 Failures: 0 Errors: 0 Skips: 0

Test Cases/TC_PATCH_UpdateUsers (3.683s)
1 - response = sendRequest(findTestObject("Object Repository/PATCH_Update
2 - verifyResponseStatusCode(response, 200) (0.025s)
3 - println(response.getResponseBodyContent()) (0.013s)

05-05-2025 03:27:57 PM Test Cases/TC_PATCH_UpdateUsers
Elapsed time: 3.683s
Test Cases/TC_PATCH_UpdateUsers

Delete Request

TC_GET_Users POST POST_CreateUser TC_POST_Cre... PATCH PATCH_Updat... TC_PATCH_Up... DELETE DELETE_Users x Help

DELETE \$(baseUrl)/users/2

Follow redirects Customize API methods Set timeout and response size limit

Query Parameters

Name	Value
------	-------

Authorization HTTP Header HTTP Body Validation Verification Variables Variables Editor

Add Remove

Name	Value
x-api-key	reqres-free-v1

Response

Status: 204 No Content Elapsed: 958 ms Size: 1008 bytes

Body Header Verification Log Validation Log

pretty raw preview

The screenshot displays a REST client interface with a DELETE request configured for the endpoint `DELETE_User`. The request steps are:

- 1 - Send Request
- 2 - Verify Response Status Code (response, 204)
- 3 - Method Call Statement (`println("Delete successful: " + response.getStatusCode())`)

The Job Progress panel on the right shows the test case `Test Cases/TC-DELETE_Users - 20250505_153440` with a status of `<Passed>`.

The Log Viewer at the bottom shows the execution details:

- Test Cases/TC-DELETE_Users (3.461s)
- 1 - response = sendRequest(findTestObject("Object Repository/DELETE_User"))
- 2 - verifyResponseStatusCode(response, 204) (0.027s)
- 3 - println("Delete successful: " + response.getStatusCode()) (0.023s)

The elapsed time for the test case is 3.461s.

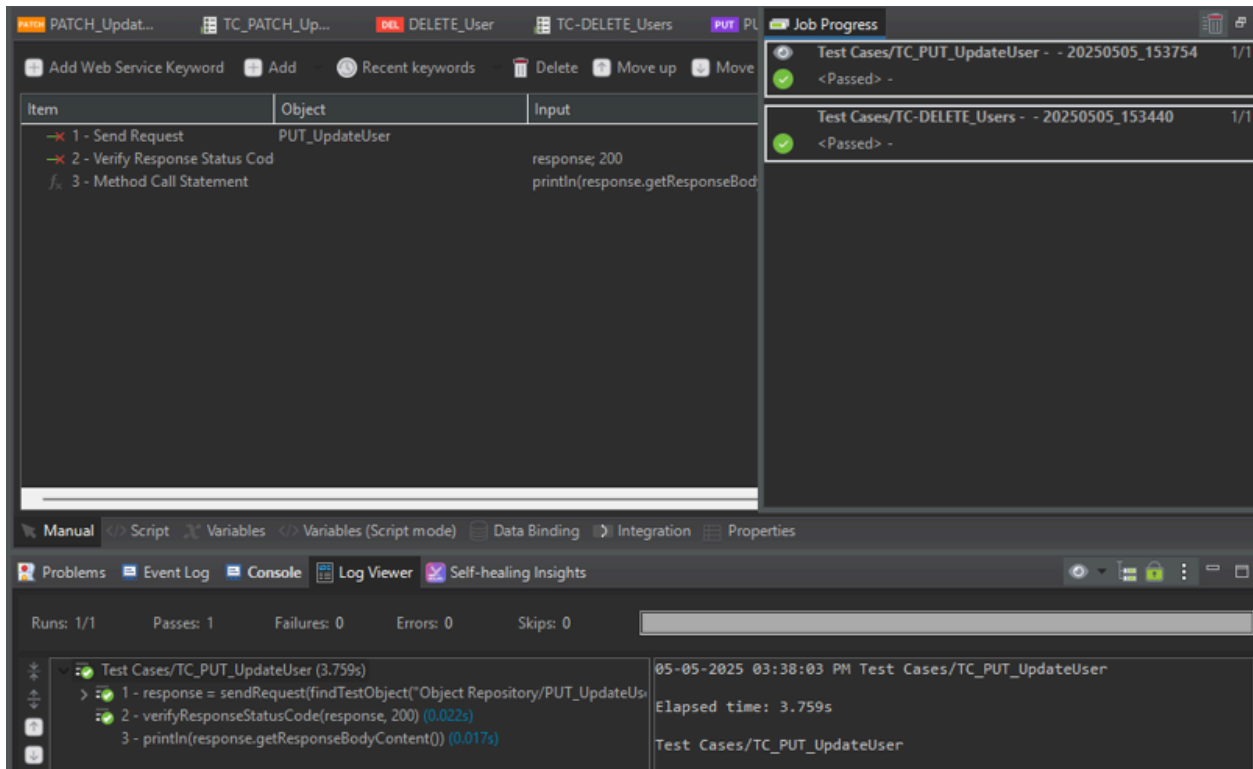
Put Request

The screenshot displays a REST client interface with a PUT request configured for the endpoint `PUT UpdateUser`. The request is set to `PUT` with the URL `$(baseUrl)/users/2`. The request body is empty.

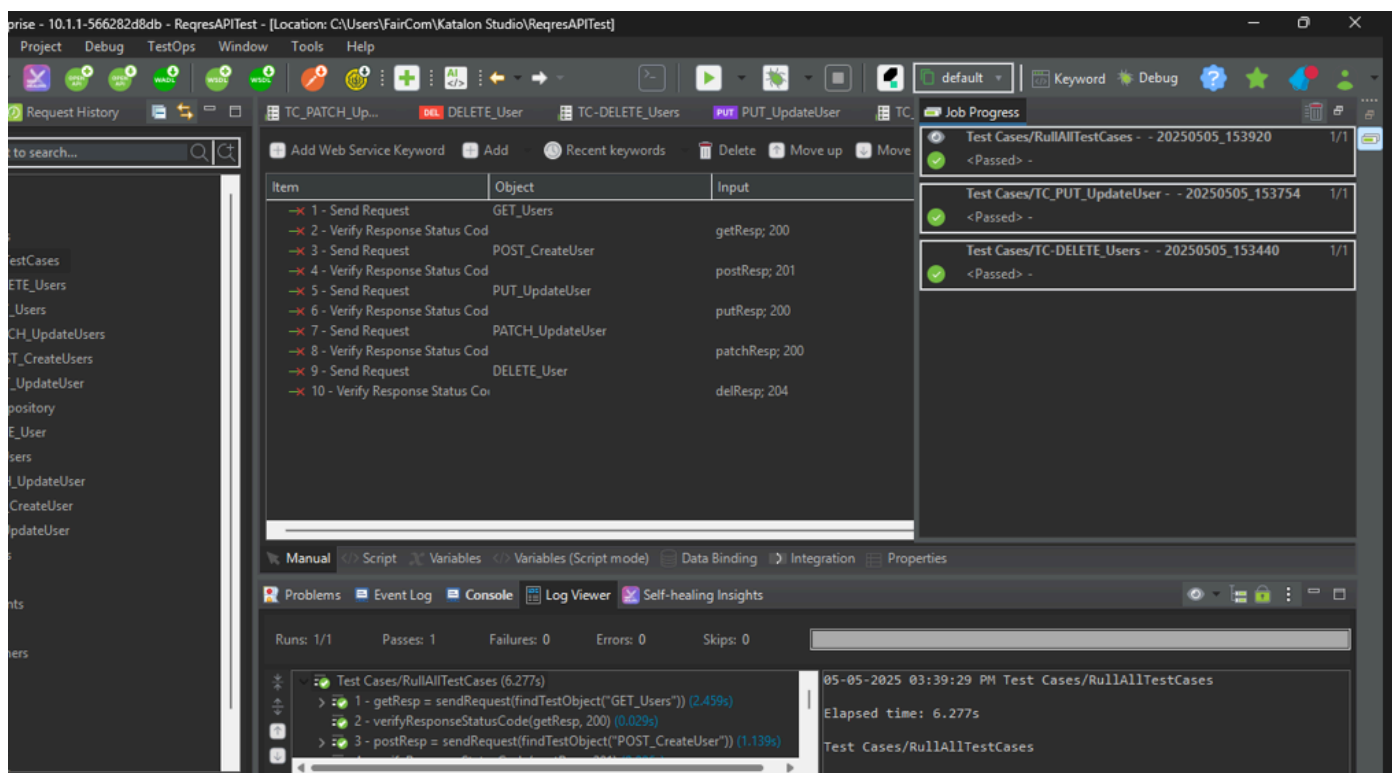
The Response panel on the right shows the status `200 OK` with an elapsed time of 954 ms and a size of 1 KB. The response body is displayed in pretty format:

```
1 {
2   "name": "morpheus",
3   "job": "zion resident",
4   "updatedAt": "2025-05-
5   05T10:36:53.206Z"
6 }
```

The Variables panel at the bottom shows a variable `baseUrl` of type `String` with a default value of `"https://reqres.in/api"`.



RunAll_TestCases



2. State all status codes along with query types.

Status Codes with Query Types

Here is a list of status codes and their meanings for each HTTP method used:

HTTP Method	Status Code	Meaning
GET	200 OK	The request was successful, and the data was returned.
POST	201 Created	A new resource was created successfully.
PUT	200 OK	The resource was successfully updated.
PATCH	200 OK	The resource was successfully partially updated.
DELETE	204 No Content	The resource was successfully deleted.